

MATA KULIAH STATISTIKA DAN PROBABILITAS

STATISTICAL AUDIT OF PANDAS

Dosen Pengampu: Muhammad Ridho Kurniawan Pratama, S.Kom., M.T.I.



Intelligentia - Dignitas

Disusun Oleh Kelompok 8:

Nama	NIM
Muhamad Fharis Arradhien	1519625010
Adhinda Zahra Dinanti	1519625016
Nayla Andhini Novia Dewani	1519625048
Kevindra Raditya Luthfiansyah	1519625004
Bram Radhitya Riezky Prayoga	1519625039

SISTEM DAN TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS NEGERI JAKARTA

2026

RINGKASAN

Dalam laporan ini kami membuat analisis statistik kesehatan terhadap repositori open-source pandas di GitHub dengan menggunakan 8.150 data yang terdiri atas 5.650 issue dan 2.500 pull request (PR) selama periode 2020–2025. Analisis dilakukan untuk memahami karakteristik pengembangan proyek, pola kemunculan bug, serta efektivitas proses peninjauan dan penyelesaian kontribusi.

Hasil estimasi parameter menunjukkan bahwa peluang sebuah pull request berhasil di-merge mencapai 69,00% ($\hat{\theta} = 0,6900$). Nilai ini didukung oleh interval kepercayaan 95% sebesar $[0,6716; 0,7078]$, yang menunjukkan bahwa estimasi tersebut cukup stabil dan memiliki tingkat ketidakpastian yang relatif kecil. Selain itu, rata-rata jumlah bug yang dilaporkan selama periode pengamatan adalah 45,67 bug per bulan ($\hat{\lambda} = 45,6727$), dengan interval kepercayaan 95% sebesar $[43,8867; 47,4588]$.

Pada tahap pengujian hipotesis, ditemukan adanya perubahan yang signifikan setelah rilis pandas 2.0 pada April 2023. Sebelum rilis, rata-rata jumlah bug yang dilaporkan mencapai 54,14 bug per bulan, sedangkan setelah rilis angka tersebut turun menjadi 40,03 bug per bulan. Hasil uji statistik menunjukkan bahwa penurunan ini tidak terjadi secara kebetulan ($z = 2,6560$; $p = 0,0079$), sehingga dapat disimpulkan bahwa rilis tersebut berkaitan dengan berkurangnya jumlah bug yang muncul.

Analisis juga dilengkapi dengan beberapa pendekatan komputasional. Simulasi Monte Carlo memperkirakan peluang sebuah issue memerlukan waktu lebih dari 30 hari untuk ditutup sebesar 30,33%, nilai yang sangat dekat dengan hasil observasi langsung sebesar 30,27%. Sementara itu, Bloom Filter mampu mendeteksi kemungkinan duplikasi issue dengan tingkat false positive sebesar 3,50%. Pada simulasi pengelolaan pull request, metode MCMC Knapsack berhasil memilih 11 PR dengan total prioritas 82 dalam batas kapasitas reviewer sebesar 20 jam kerja per minggu.

Secara keseluruhan, hasil analisis menunjukkan bahwa kombinasi metode statistika inferensial dan komputasional dapat memberikan gambaran yang lebih mendalam mengenai dinamika pengembangan perangkat lunak pada proyek open-source. Temuan-temuan yang didapat tidak hanya membantu memahami kondisi repositori selama periode pengamatan, tetapi juga dapat menjadi dasar dalam pengambilan keputusan dan evaluasi proses pengembangan kedepannya.

GAMBARAN UMUM DATA

Sumber Data

Data pada project ini diambil dari repositori open-source pandas-dev/pandas di GitHub menggunakan GitHub REST API dan GitHub Search API. Pengambilan data dilakukan secara otomatis melalui script `fetch_data.py` dengan cakupan periode 2020 hingga 2025. Terdapat dua jenis data yang dikumpulkan:

- Issues diambil via Search API dengan filter `is:issue is:closed`, menghasilkan data permasalahan yang sudah ditutup. Pengambilan dilakukan per tahun untuk menghindari batas hasil pencarian dari API.
- Pull Requests (PR) diambil via endpoint `/repos/pandas-dev/pandas/pulls` dengan filter `state: closed`, menghasilkan data PR yang sudah ditutup maupun di-merge.

Data mentah disimpan dalam format JSON (`issues_raw.json` dan `pulls_raw.json`) di folder `data/raw/`.

Proses Pembersihan Data

Script `clean_data.py` mengolah data mentah menjadi dataset siap analisis. Proses yang dilakukan meliputi:

- Parsing dan seleksi kolom hanya kolom yang relevan yang dipertahankan, yaitu: `id`, `type`, `state`, `created_at`, `closed_at`, `days_to_close`, `label_names`, `is_bug`, `is_merged`, `year_month`, dan `year`.
- Kalkulasi durasi kolom `days_to_close` dihitung dari selisih `closed_at` minus `created_at` dalam satuan hari.
- Ekstraksi label label-label dari setiap item digabungkan menjadi string, dan kolom `is_bug` diberi nilai 1 jika label mengandung kata "bug".
- Penandaan merge khusus PR, kolom `is_merged` bernilai 1 jika field `merged_at` tidak kosong.
- Pembersihan anomali baris dengan nilai `days_to_close` yang kosong (null) atau negatif dihapus.

Ukuran Dataset

Berdasarkan output dari notebook EDA (`01_eda.ipynb`), dataset yang digunakan memiliki ukuran sebagai berikut :

1. Dataset.csv : 8.150 Jumlah Baris
2. Issues.csv : 5.650 baris
3. Pulls.csv : 2.500 baris

Total terdapat **11 kolom** pada setiap dataset.

Struktur dan Deskripsi Kolom

Dataset memiliki 11 kolom yang masing-masing merepresentasikan atribut berbeda dari setiap issue maupun pull request.

Kolom `id` berisi nomor unik yang mengidentifikasi setiap issue atau PR di GitHub. Kolom `type` menunjukkan jenis item, apakah berupa issue atau `pull_request`. Kolom `state` mencatat status item tersebut, yaitu `open` atau `closed`.

Untuk informasi waktu, terdapat kolom `created_at` yang menyimpan timestamp pembuatan item dan `closed_at` yang menyimpan timestamp penutupannya, keduanya dalam format UTC. Dari kedua kolom waktu tersebut, diturunkan kolom `days_to_close` yang merepresentasikan durasi penyelesaian item dalam satuan hari.

Kolom `label_names` menyimpan seluruh label yang melekat pada sebuah item, digabungkan dalam satu string yang dipisahkan koma. Dari kolom ini diturunkan kolom `is_bug`, sebuah flag bernilai 1 jika item memiliki label yang mengandung kata "bug", dan 0 jika tidak. Khusus untuk pull request, terdapat kolom `is_merged` yang bernilai 1 apabila PR berhasil di-merge, dan 0 jika hanya ditutup tanpa di-merge.

Terakhir, terdapat dua kolom turunan waktu untuk keperluan agregasi, yaitu `year_month` dalam format YYYY-MM dan `year` yang hanya menyimpan tahun pembuatan item.

Variabel Kunci untuk Analisis

Dari hasil EDA, ditemukan beberapa nilai variabel penting yang menjadi dasar analisis selanjutnya:

- Merge rate PR -> proporsi PR dengan `is_merged = 1`, digunakan sebagai input estimasi MLE Bernoulli.
- Median `days_to_close` -> digunakan sebagai threshold untuk analisis probabilitas issue > 30 hari.
- Rata-rata bug/bulan sebelum pandas 2.0 -> 54,14 bug/bulan (sebelum April 2023).
- Rata-rata bug/bulan setelah pandas 2.0 -> 40,03 bug/bulan (mulai April 2023), menunjukkan adanya penurunan setelah rilis versi mayor.

Titik pemisah yang digunakan adalah April 2023, sesuai tanggal rilis pandas versi 2.0.

ESTIMASI PARAMETER

Tujuan dan Pendekatan

Pada bagian ini bertujuan menjawab dua research question utama menggunakan metode estimasi statistik, khususnya Maximum Likelihood Estimation (MLE). MLE adalah metode untuk mencari nilai parameter yang paling "masuk akal" atau paling mungkin menghasilkan data yang sudah diamati. Dua distribusi yang digunakan disesuaikan dengan karakteristik data masing-masing research question.

Implementasi Fungsi Estimasi

Seluruh fungsi estimasi diimplementasikan secara modular dalam file estimator.py agar dapat digunakan ulang di notebook-notebook selanjutnya.

1. MLE Bernoulli (`mle_bernoulli`) digunakan untuk mengestimasi probabilitas sebuah PR akan di-merge. Fungsi ini menerima array biner (0/1), menghitung jumlah PR yang berhasil di-merge (k) dari total PR (n), lalu mengembalikan nilai $\hat{\theta} = k/n$ sebagai estimasi parameter. Fungsi ini juga memvalidasi bahwa data hanya berisi nilai 0 dan 1.
2. MLE Poisson (`mle_poisson`) digunakan untuk mengestimasi rata-rata jumlah bug report per bulan. Pada distribusi Poisson, nilai MLE-nya adalah rata-rata aritmetika sederhana dari data, sehingga $\hat{\lambda} = \text{mean}(\text{data})$. Fungsi ini juga memvalidasi bahwa data tidak mengandung nilai negatif.
3. Beta Posterior (`beta_posterior`) digunakan untuk analisis Bayesian pada merge rate PR. Dengan menggunakan prior seragam (uniform prior), posterior dari parameter Bernoulli mengikuti distribusi Beta dengan parameter $\alpha = k + 1$ dan $\beta = m + 1$, di mana k adalah jumlah PR yang di-merge dan m adalah jumlah yang ditolak. Fungsi ini mengembalikan mean dan modus dari distribusi posterior tersebut.
4. Log-Likelihood (`log_likelihood_bernoulli` dan `log_likelihood_poisson`) diimplementasikan untuk keperluan visualisasi kurva log-likelihood, sehingga dapat terlihat secara grafis di titik mana nilai MLE berada sebagai puncak kurva.

Hasil Estimasi RQ1 PR Merge Rate (MLE Bernoulli)

Dari total 2.500 pull request, sebanyak 1.725 PR berhasil di-merge dan 775 PR ditolak. Dari data ini diperoleh:

Nilai $\hat{\theta}$ (MLE) = 0,6900, artinya probabilitas estimasi sebuah PR akan di-merge adalah sekitar 69,00%.

Untuk mengukur ketidakpastian estimasi tersebut, dihitung Confidence Interval (CI) 95% menggunakan dua metode. Metode Wald menghasilkan interval [0,6719 — 0,7081], sedangkan metode Clopper–Pearson yang lebih akurat untuk data biner menghasilkan interval yang hampir identik yaitu [0,6715 — 0,7081]. Sempitnya interval ini menunjukkan estimasi yang cukup presisi karena jumlah sampel yang besar.

Pada analisis Bayesian dengan Beta Posterior, diperoleh mode posterior sebesar 0,6900 dan mean posterior sebesar 0,6898 — keduanya sangat dekat dengan nilai MLE, yang konsisten karena jumlah data yang besar membuat pengaruh prior menjadi sangat kecil.

Hasil Estimasi RQ2 Bug Report Rate (MLE Poisson)

Data bug dikelompokkan per bulan dari issues.csv, menghasilkan deret waktu jumlah bug per bulan. Dari observasi selama beberapa puluh bulan tersebut, fungsi `mle_poisson` menghitung $\hat{\lambda}$ sebagai rata-rata jumlah bug per bulan secara keseluruhan. Nilai $\hat{\lambda}$ ini nantinya digunakan sebagai λ referensi pada tahap hypothesis testing untuk membandingkan rata-rata bug sebelum dan sesudah rilis pandas 2.0 (April 2023).

Output untuk Tahap Selanjutnya

Hasil estimasi pada notebook ini menjadi input untuk analisis di tahap berikutnya. Nilai $\hat{\theta}$ dari MLE Bernoulli digunakan sebagai nilai H_0 pada uji hipotesis proporsi merge rate. Nilai $\hat{\lambda}$ dari MLE Poisson digunakan sebagai λ referensi pada Z-test satu sampel untuk menguji perubahan rata-rata bug report. Adapun parameter distribusi Beta Posterior (α dan β) digunakan sebagai prior pada simulasi Bayesian di tahap simulasi Monte Carlo.

INTERVAL KEPERCAYAAN

Setelah parameter diestimasi menggunakan metode Maximum Likelihood Estimation (MLE) pada tahap sebelumnya, langkah selanjutnya adalah mengevaluasi seberapa pasti hasil estimasi tersebut. Karena nilai yang diperoleh dari sampel belum tentu sama persis dengan parameter populasi yang sebenarnya, diperlukan suatu ukuran ketidakpastian dalam bentuk interval estimasi. Interval ini menunjukkan rentang nilai yang diperkirakan memuat parameter sebenarnya dengan tingkat keyakinan tertentu.

Pada penelitian ini digunakan dua pendekatan yang berbeda, yaitu Confidence Interval yang berasal dari pendekatan frequentist dan Credible Interval yang berasal dari pendekatan Bayesian. Keduanya memiliki tujuan yang sama, yaitu memberikan informasi mengenai ketidakpastian estimasi parameter, namun didasarkan pada interpretasi probabilitas yang berbeda.

Seluruh fungsi yang digunakan untuk menghitung interval tersebut, yaitu `ci_bernoulli`, `ci_poisson`, dan `credible_interval`, Perhitungan Confidence Interval dan Credible Interval dilakukan menggunakan kumpulan fungsi yang terdapat pada file `src/inference.py`. Perhitungan Confidence Interval dilakukan menggunakan fungsi dasar `confidence_interval`, yang menerapkan pendekatan Central Limit Theorem (CLT) untuk menentukan rentang estimasi parameter yaitu: $\hat{\theta} \pm z \times (\sigma / \sqrt{n})$, di mana $\hat{\theta}$ merupakan estimasi parameter, σ adalah simpangan baku, n adalah ukuran sampel, dan z adalah nilai kuantil dari distribusi normal standar yang sesuai dengan tingkat kepercayaan yang dipilih.

Dari total 2.500 pull request yang dianalisis, sebanyak 1.725 PR berhasil di-merge, menghasilkan estimasi MLE $\hat{\theta} = 0,6900$.

A. Confidence Interval

Untuk mengukur ketidakpastian estimasi ini, dihitung Confidence Interval menggunakan fungsi `ci_bernoulli` yang mengestimasi standard deviasi dari distribusi Bernoulli sebagai $\sigma = \sqrt{\hat{\theta} \cdot (1 - \hat{\theta})}$.

Hasil yang diperoleh pada tiga level kepercayaan :

CI 90% menghasilkan interval [0,6748; 0,7052],

CI 95% menghasilkan interval [0,6719; 0,7081],

CI 99% menghasilkan interval [0,6662; 0,7138].

B. Credibel Interval Bayesian

Analisis Bayesian dilakukan menggunakan distribusi posterior Beta yang diperoleh dari fungsi `beta_posterior`. Dengan menggunakan uniform prior (prior seragam yang tidak memihak nilai θ tertentu), posterior mengikuti distribusi Beta(α, β) di mana $\alpha = k + 1 = 1.726$ dan $\beta = m + 1 = 776$. Parameter posterior ini kemudian dimasukkan ke fungsi `credible_interval` yang menghitung batas interval menggunakan fungsi kuantil distribusi Beta.

Hasil yang diperoleh pada tiga level kepercayaan:

CrI 90% menghasilkan interval [0,6746; 0,7050],

CrI 95% menghasilkan interval [0,6716; 0,7078],

CrI 99% menghasilkan interval [0,6658; 0,7134].

Mean posterior yang diperoleh adalah 0,6898 dan mode posterior adalah 0,6900. Berbeda dengan confidence interval, credible interval memiliki interpretasi probabilistik langsung, terdapat probabilitas sebesar 95% bahwa nilai θ yang sebenarnya berada di dalam rentang

[0,6716;0,7078], berdasarkan data yang telah diamati dan prior yang digunakan. Hal ini juga berlaku untuk CrI 90% dan 99% masing-masing menyatakan bahwa terdapat probabilitas 90% dan 99% bahwa θ berada dalam rentang interval yang diperoleh.

C. Interval Estimasi Bug Report Rate (Poisson)

Data bug diperoleh dari file issues.csv dan kemudian dikelompokkan berdasarkan bulan menggunakan. Proses ini menghasilkan deret waktu yang menunjukkan jumlah bug yang ditemukan setiap bulan selama periode 2020-2025. Untuk mengestimasi parameter distribusi Poisson, fungsi `ci_poisson` menghitung nilai $\hat{\lambda}$ (lambda hat) sebagai rata-rata jumlah bug per bulan. Karena pada distribusi Poisson nilai rata-rata dan varians adalah sama, simpangan baku dapat dihitung menggunakan rumus $\sigma = \sqrt{\hat{\lambda}}$.

Berdasarkan data yang dianalisis, diperoleh estimasi Maximum Likelihood Estimation (MLE) sebesar $\hat{\lambda} = 45,6727$ bug per bulan. Dengan tingkat kepercayaan 95%, diperoleh Confidence Interval (CI) sebesar [43,8867, 47,4588] dengan margin of error $\pm 1,7861$. Sementara itu, pada tingkat kepercayaan 99%, interval yang dihasilkan menjadi lebih lebar, yaitu [43,3255, 48,0200].

Nilai $\hat{\lambda} = 45,6727$ beserta interval kepercayaannya memberikan gambaran umum mengenai rata-rata jumlah bug yang terjadi selama periode pengamatan. Estimasi ini akan digunakan sebagai nilai acuan (baseline) pada tahap pengujian hipotesis yang dilakukan oleh Member D.

UJI HIPOTESIS

Uji 1: One-Sample Poisson Z-Test (Validasi Baseline Target)

1. Formulasi Hipotesis

- **Hipotesis Nol** (H_0): $\lambda = \lambda_0$
Rata-rata kemunculan bug per bulan setelah rilis Pandas 2.0 sama dengan target nilai standar baseline historis yang ditentukan kelompok (λ_0).
- **Hipotesis Alternatif** $\lambda \neq \lambda_0$
Rata-rata kemunculan bug per bulan setelah rilis Pandas 2.0 berbeda secara signifikan (dua arah) dari target baseline standar historis.

2. Tingkat Signifikansi (Significance Level)

- $\alpha = 0.05$ (Taraf nyata 5% dengan pengujian dua arah, sehingga nilai kritis ambang batas penolakan berada pada posisi baku $z < -1.96$ atau $z > 1.96$).

3. Statistik Uji (Test Statistic)

Menggunakan metode uji statistik parametrik Poisson Z-Test satu sampel mandiri:

$$Z = \frac{\hat{\lambda} - \lambda_0}{\sqrt{\lambda_0/n}}$$

4. Daerah Kritis (Critical Region)

Keputusan penolakan dilakukan apabila nilai statistik hitung memenuhi kriteria:

- Tolak H_0 jika $Z_{hitung} < -1.96$ atau $Z_{hitung} > 1.96$, atau nilai probabilitas kesalahan taksir ($p\text{-value} < \alpha$).

5. Perhitungan Nilai Aktual & p-value

- Nilai Z -stat Aktual: -3.9870
- Nilai p -value Aktual: 0.0001

6. Keputusan Statistik & Kesimpulan Contextual

- **Keputusan: Tolak H_0** karena nilai kuantitatif dari $Z_{hitung} = -3.9870$ jatuh jauh di dalam batas ekstrem daerah kritis penolakan kiri, didukung secara mutlak oleh perolehan tingkat signifikansi nilai probabilitas $p\text{-value} = 0.0001$ yang jauh lebih kecil dari standar batas toleransi kekeliruan alfa (0.05).
- **Kesimpulan:** Terdapat bukti ilmiah yang sangat kuat dan valid pada tingkat kepercayaan 95% untuk menyatakan bahwa rata-rata tingkat kemunculan bug bulanan pasca implementasi pembaruan Pandas 2.0 mengalami pergeseran matematis secara nyata dari acuan dasar target historis. Hal ini mengonfirmasi performa sistem mengalami dinamika perubahan stabilitas kode.

Uji 2: Two-Sample Poisson Z-Test (Komparasi Sebelum vs Sesudah Pandas 2.0)

1. Formulasi Hipotesis

- **Hipotesis Nol** $(H_0): \lambda_{before} = \lambda_{after}$
Tidak ada perbedaan rata-rata tingkat kemunculan bug per bulan antara periode waktu sebelum rilis Pandas 2.0 dengan periode waktu sesudah rilis Pandas 2.0.
- **Hipotesis Alternatif** $(H_1): \lambda_{before} \neq \lambda_{after}$
Terdapat perbedaan rata-rata tingkat kemunculan bug per bulan yang signifikan secara statistik antara pengamatan periode sebelum dan sesudah rilis pembaruan Pandas 2.0.

2. Tingkat Signifikansi (Significance Level)

- $\alpha = 0.05$ (Taraf nyata dua arah dengan tingkat signifikansi standar industri, menetapkan nilai ambang penolakan batas kritis pada tabel sebesar $Z = \pm 1.96$).

3. Statistik Uji (Test Statistic)

Menggunakan formulasi uji komparatif independen dua sampel berpasangan berbasis sebaran Poisson:

$$Z = \frac{\hat{\lambda}_1 - \hat{\lambda}_2}{\sqrt{\frac{\hat{\lambda}_1}{n_1} + \frac{\hat{\lambda}_2}{n_2}}}$$

4. Daerah Kritis (Critical Region)

- Tolak hipotesis nol jika akumulasi absolut nilai hitung $|Z_{hitung}| > 1.96$ atau apabila nilai taraf nyata empiris ($p\text{-value} < 0.05$).

5. Perhitungan Nilai Aktual & $p\text{-value}$

- Nilai $Z\text{-stat}$ Aktual: 2.6560
- Nilai $p\text{-value}$ Aktual: 0.0079

6. Keputusan Statistik & Kesimpulan Contextual

- **Keputusan: Tolak H_0** karena nilai aktual hitung kuantitas konvergensi statistik diperoleh $Z_{hitung} = 2.6560$ yang bernilai lebih besar dari ambang batas kritis sebaran positif yaitu 1.96. Keputusan didukung penuh oleh temuan probabilitas nilai korelasi $p\text{-value} = 0.0079 < 0.05$.
- **Kesimpulan:** Secara meyakinkan dapat disimpulkan bahwa volume sebaran kemunculan rilis bug bulanan terbukti berubah secara signifikan setelah implementasi perubahan arsitektur versi Pandas 2.0. Ketidaksamaan matematis konvergen ini memberikan legitimasi ilmiah bagi tahap pemodelan simulasi selanjutnya (**Member E — Simulation**) untuk memisahkan parameter fungsi pembangkit data distribusi stokastik (tidak boleh digabung).

Ringkasan Hasil Parameter Uji

Variabel	Nilai	Digunakan di
z-stat Uji 1 (One-Sample Poisson Z-test)	-3.9870	Notebook 04
p-value Uji 1 (One-Sample Poisson Z-test)	0.0001	Notebook 04
z-stat Uji 2 (Two-Sample Poisson Z-test)	2.6560	Notebook 04, Notebook 05
p-value Uji 2 (Two-Sample Poisson Z-test)	0.0079	Notebook 04

Output Rekomendasi Mandat untuk Role Selanjutnya (Member E - Simulation):

1. **Parameter Generator Baru:** Gunakan nilai parameter laju empiris aktual setelah rilis yaitu $\lambda_{after} = 40.0303$ sebagai basis nilai *rate parameter* tunggal dalam pemodelan sebaran penarikan sampel fungsi stokastik simulasi ke depan.
2. **Pemisahan Model:** Keputusan mutlak **Tolak H_0** membuktikan secara konklusif bahwa performa sebelum vs sesudah rilis memiliki karakteristik yang berbeda secara signifikan, sehingga simulasi wajib memisahkan kedua skenario tersebut dalam model statistik terpisah.

ANALISIS KOMPUTASIONAL

Pada bagian ini digunakan tiga pendekatan komputasional, yaitu Monte Carlo Simulation, Bloom Filter, dan MCMC Knapsack. Ketiga metode dipilih karena mampu membantu menjawab permasalahan yang tidak mudah diselesaikan hanya dengan perhitungan statistik dasar.

Monte Carlo Simulation

Monte Carlo Simulation digunakan untuk mengestimasi probabilitas sebuah issue membutuhkan waktu lebih dari 30 hari untuk ditutup. Metode ini bekerja dengan melakukan pengambilan sampel secara acak dari distribusi data yang telah diamati sebanyak 100.000 kali.

Hasil simulasi menunjukkan probabilitas sebesar 0,3033 atau 30,33%. Nilai ini sangat dekat dengan nilai empiris aktual sebesar 30,27%, dengan selisih hanya 0,0006. Hal ini menunjukkan bahwa simulasi Monte Carlo mampu merepresentasikan pola data dengan baik dan memberikan estimasi yang stabil. Metode Monte Carlo dipilih karena mampu memperkirakan probabilitas suatu kejadian tanpa harus menggunakan rumus analitik yang kompleks.

Bloom Filter

Bloom Filter digunakan untuk mensimulasikan proses deteksi duplikasi issue secara efisien. Pada implementasi ini digunakan 5.650 issue, 3 fungsi hash, dan bit array berukuran 50.000.

Hasil pengujian menunjukkan false positive rate empiris sebesar 3,50%. Artinya terdapat kemungkinan kecil sistem menganggap suatu issue sudah pernah muncul padahal sebenarnya belum pernah ada. Meskipun demikian, Bloom Filter tetap sangat berguna karena mampu melakukan pengecekan dengan penggunaan memori yang rendah dan kecepatan yang tinggi. Metode ini dipilih karena sesuai untuk proses pemeriksaan awal terhadap data berukuran besar yang membutuhkan efisiensi penyimpanan.

MCMC Knapsack

MCMC Knapsack digunakan untuk membantu proses alokasi waktu reviewer pada pull request yang tersedia. Simulasi dilakukan terhadap 20 pull request dengan kapasitas reviewer sebesar 20 jam per minggu.

Hasil simulasi menunjukkan bahwa algoritma memilih 11 pull request dengan total nilai prioritas sebesar 82 dan menggunakan seluruh kapasitas reviewer yang tersedia, yaitu 20 dari 20 jam. Acceptance rate yang diperoleh sebesar 0,6578 menunjukkan bahwa proses pencarian solusi berjalan dengan baik dan mampu mengeksplorasi berbagai kombinasi kandidat. Metode MCMC Knapsack dipilih karena mampu mencari kombinasi pull request yang memberikan nilai prioritas tinggi tanpa melampaui keterbatasan waktu reviewer.

Evaluasi Metode

Ketiga metode yang digunakan memiliki fungsi yang berbeda-beda. Monte Carlo digunakan untuk memperkirakan peluang suatu kejadian berdasarkan data yang sudah ada. Bloom Filter

digunakan untuk membantu mendeteksi kemungkinan data yang duplikat dengan cepat dan menggunakan memori yang lebih sedikit. Sementara itu, MCMC Knapsack digunakan untuk membantu memilih pekerjaan yang paling penting ketika waktu atau sumber daya yang tersedia terbatas.

Dari hasil simulasi yang dilakukan, ketiga metode tersebut mampu memberikan informasi yang berguna dan dapat membantu proses pengambilan keputusan dalam pengelolaan proyek *open-source* seperti pandas.

REKOMENDASI

Berdasarkan hasil audit statistik dan analisis komputasional yang telah dilakukan, tim merekomendasikan beberapa tindakan strategis untuk mengoptimalkan manajemen repositori Pandas. Pertama, manajemen alokasi sumber daya pengembang pasca-rilis Pandas 2.0 harus disesuaikan menggunakan parameter laju *bug* yang baru, yaitu $\lambda = 40,03$ bug per bulan. Penurunan volume *bug* yang signifikan ini membuktikan keberhasilan arsitektur baru, sehingga tim pengembang dapat mengalihkan sebagian fokus mereka dari perbaikan *bug* (*reactive maintenance*) ke pengembangan fitur baru atau refaktorisasi kode (*proactive development*). Namun, pemisahan model evaluasi antara periode sebelum dan sesudah rilis harus tetap dipertahankan dalam metrik internal untuk memastikan fluktuasi stabilitas kode dapat dipantau secara akurat tanpa bias data historis lama.

Kedua, untuk mengatasi kendala operasional terkait durasi penyelesaian *issue*, tim manajemen proyek disarankan mengintegrasikan sistem peringatan dini (*early-warning system*) berbasis simulasi Monte Carlo. Mengingat probabilitas sebuah *issue* tertahan lebih dari 30 hari mencapai 30,33%, sistem otomatis dapat diprogram untuk memberikan anotasi khusus atau menaikkan tingkat prioritas pada *issue* yang telah terbuka mendekati batas waktu kritis tersebut. Langkah ini penting untuk menjaga kepuasan komunitas pengguna dan memastikan permasalahan yang kompleks tidak terbengkalai.

Ketiga, otomatisasi infrastruktur repositori perlu ditingkatkan melalui implementasi algoritma advanced yang telah diuji dalam penelitian ini. Penggunaan *Bloom Filter* direkomendasikan sebagai penyaring pertama (*first-line defense*) pada bot GitHub untuk mendeteksi duplikasi *issue* secara instan. Dengan tingkat *false positive* yang rendah (3,50%), metode ini akan sangat menghemat kapasitas memori peladen dan waktu pemrosesan sebelum *issue* diperiksa secara manual. Sementara itu, untuk mengatasi keterbatasan waktu peninjau (*reviewer workflow*), tim dapat menerapkan modul penjadwalan otomatis berbasis *MCMC Knapsack*. Algoritma ini terbukti mampu mengoptimalkan pemanfaatan 100% kapasitas waktu *reviewer* (20 jam/minggu) untuk menyelesaikan kombinasi *pull request* yang memiliki nilai prioritas tertinggi, sehingga proses merger kode menjadi lebih efisien dan terarah.